

CHALLENGE | FORD

Desafio 01 — Inteligência Competitiva Automotiva

Data de entrega: 24/05/2026

SpecRadar

Plataforma de inteligência competitiva automotiva com extração semântica e repositório histórico acumulativo

Matéria	Responsáveis
Arquitetura e Web Services	Renan e Camila
Mobile Development and IoT	Isabelle e Pedro
Testing, Compliance and QA	Todos (vídeo pitch: Isabelle)
Cybersecurity	Renan e Camila
Inteligência Artificial & ML	Nicoli e Claudio

Resumo do Projeto

Desafio: Desafio 01 — Inteligência Competitiva Automotiva

Problema: Coletar especificações técnicas de veículos concorrentes consome ~1 hora por versão de forma manual, com alta probabilidade de imprecisão. O analista navega em sites, assiste vídeos, liga para concessionárias e preenche planilhas à mão — que ficam desatualizadas rapidamente.

Impacto no negócio: Decisões de posicionamento de produto baseadas em dados imprecisos ou desatualizados afetam a estratégia comercial e a argumentação de vendas da Ford frente à concorrência.

Solução

O SpecRadar é uma plataforma de inteligência competitiva automotiva que elimina o gargalo de coleta manual. O analista consulta qualquer veículo concorrente em linguagem natural ou via formulário estruturado e recebe em segundos uma ficha técnica padronizada, sempre no mesmo formato.

Dois modos de interação sobre a mesma base técnica:

- Modo chat: consultas em linguagem natural e cruzamento de dados já no repositório histórico (ex: 'Qual pick-up concorrente tem mais torque abaixo de R\$ 300 mil?')
- Modo formulário: entrada estruturada de Marca, Modelo, Versão e lista de atributos — exatamente como o brief da Ford especifica — com saída em tabela exportável

Diferencial central:

- Confidence score por campo: cada spec retorna com confiança (alta / inferida / não encontrado), URL da fonte e data de verificação — nenhum sistema de dados automotivos existente oferece isso
- Repositório histórico acumulativo: cada consulta enriquece o banco coletivo, permitindo comparações instantâneas sem nova busca
- Sensor de demanda preditiva: monitora menções públicas de veículos concorrentes e atualiza o banco preventivamente antes das perguntas chegarem

Objetivo

- Reduzir de ~60 min para menos de 10 segundos o tempo de coleta de specs por versão
- Garantir padronização e rastreabilidade de todos os dados coletados
- Construir repositório comparativo permanente de especificações da concorrência
- Validação: sistema deve entregar corretamente todas as specs da Ford Ranger Raptor (caso de teste oficial do brief)



Status Geral das Tarefas

Tarefa	Responsável	Matéria	Status
Documentação Swagger / README da API	Renan e Camila	Web Services	Não iniciado
Implementação da API RESTful de extração	Renan e Camila	Web Services	Não iniciado
Endpoint de comparação entre veículos	Renan e Camila	Web Services	Não iniciado
Confidence score + verificado_em por campo	Renan e Camila	Web Services	Não iniciado
App React Native — tela de consulta (chat)	Isabelle e Pedro	Mobile & IoT	Não iniciado
App React Native — tela formulário + histórico	Isabelle e Pedro	Mobile & IoT	Não iniciado
Tela de comparação visual lado a lado	Isabelle e Pedro	Mobile & IoT	Não iniciado
Pitch estruturado (10–15 slides)	Todos	Testing & QA	Urgente
Business Model Canvas	Todos	Testing & QA	Urgente
Quadro de Valor (stakeholders + métricas)	Todos	Testing & QA	Urgente
Arquitetura TOGAF no Archi (.archimate)	Todos	Testing & QA	Urgente
Vídeo pitch (até 3 min, todos participam)	Isabelle (liderança)	Testing & QA	Urgente
JWT + RBAC + rate limiting	Renan e Camila	Cybersecurity	Não iniciado
Sanitização de entradas + tratamento de erros	Renan e Camila	Cybersecurity	Não iniciado
Logs de auditoria + criptografia em repouso	Renan e Camila	Cybersecurity	Não iniciado
Alinhar escopo AI/ML com professor	Nicoli e Claudio	AI & ML	Urgente
Jupyter Notebook + relatório PDF	Nicoli e Claudio	AI & ML	Não iniciado
Demo ao vivo com Ranger Raptor (ensaio)	Todos	Apresentação	Não iniciado



Arquitetura Orientada a Serviços e Web Services

Responsáveis: Renan e Camila

Potencial: 7 → 9.5

O que a matéria pede

Integração por Web Services (50%)

- Desenho de arquitetura contendo os componentes que pretendem usar (10%)
- Implementação de APIs RESTful ou SOAP para comunicação entre sistemas (20%)
- Uso adequado de métodos HTTP (10%)
- Documentação das APIs com README ou Swagger como contrato com os endpoints (10%)

Arquitetura Orientada a Serviços (SOA) (20%)

- Organização modular baseada em serviços independentes e reutilizáveis (10%)
- Separação clara entre camadas de apresentação, serviço e dados (10%)

Padrões e Boas Práticas (15%)

- Adoção de padrões como REST, JSON (8%)
- Tratamento adequado de erros e exceções nos serviços (7%)

Conexão com banco de dados (15%)

- Dependências e configurações para conexão (8%)
- Controle de migrações (7%)

O que fazer no SpecRadar

1. Documentar todos os endpoints no Swagger (vale 10% — não entregar é eliminatório)

- POST /api/v1/specs/query — entrada: { marca, modelo, versao, atributos[] } | saída: ficha padronizada com confidence + fonte + verificado_em
- GET /api/v1/specs/compare?v1=X&v2=Y&atributos=a,b — retorna os dois registros do banco lado a lado com diff automático
- POST /api/v1/specs/from-pdf — upload de PDF de catálogo, retorna a ficha extraída
- GET /api/v1/specs/history — lista registros no banco com filtros de marca/modelo
- Documentar: método HTTP, path, parâmetros, body de exemplo, resposta 200 e respostas de erro (401, 404, 422, 500)

2. Separação de camadas (vale 10% — desenhar e citar no documento)

- Camada de apresentação: controllers da API (recebem requests, devolvem responses padronizados)
- Camada de serviço: lógica de extração, normalização de atributos, cálculo de confidence score, sensor de demanda
- Camada de dados: repositório de specs no banco, histórico de consultas, controle de migrações com Alembic

3. Tratamento de erros obrigatório em todos os endpoints

- Veículo não encontrado → 404 com campo 'sugestoes_similares'
- Atributo sem dado disponível → campo retornado como { valor: null, confiança: 'nao_encontrado' } — nunca omitido
- Falha de fonte externa (timeout) → 503 com mensagem genérica, sem expor tecnologia interna
- Entrada malformada → 422 com lista de campos inválidos

4. Diferencial: endpoint de comparação direta entre dois veículos

- GET /api/v1/specs/compare retorna os dois veículos do banco lado a lado
- Inclui campo 'vencedor' por atributo (qual dos dois tem valor superior)
- Valida o repositório histórico como feature real de inteligência competitiva
- Simples de implementar se o banco já existe — alto impacto na banca



O que a matéria pede

- App mobile multiplataforma (iOS e Android) com React Native + Expo
- Navegação com Expo Router
- Consumo de APIs assíncronas com tratamento de loading e erro
- Integração com pelo menos uma fonte de dados externa via API
- Diferenciais avaliados: notificações, armazenamento local, integração com sensores
- Avaliação: qualidade técnica, consistência de UX, aderência ao problema de negócio

O que fazer no SpecRadar

1. Especificar React Native + Expo Router explicitamente (obrigatório)

- Nomear a stack no documento: React Native com Expo SDK + Expo Router para navegação
- Estrutura de tabs: (1) Chat / (2) Formulário / (3) Comparar / (4) Histórico
- Cada tab é uma rota do Expo Router — mostrar a estrutura de arquivos no documento

2. Consumo assíncrono da API com tratamento completo de estados

- useState para dados, loading e erro em cada tela de consulta
- Loading spinner visível durante chamada à API
- Mensagem de erro amigável quando a API falha ou o veículo não é encontrado
- Timeout de 15s com feedback ao usuário se a extração demorar

3. Armazenamento local com AsyncStorage (diferencial avaliativo)

- Salvar últimas 10 consultas no AsyncStorage do dispositivo
- Tela de histórico offline — funciona sem conexão
- Útil em concessionárias com internet instável — argumento de negócio real

4. Tela de comparação visual lado a lado (maior diferencial do app)

- Selecionar dois veículos do histórico e comparar atributos em tabela lado a lado
- Células coloridas: verde para o veículo com valor superior em cada atributo
- Demonstra domínio de componentes React Native + estado + API em uma tela só
- Conecta diretamente ao endpoint GET /specs/compare da API

5. Consistência de UX e narrativa clara de produto

- Tela inicial com busca rápida proeminente — o analista vê o valor em 5 segundos
- Ficha de specs com indicador visual de confidence: verde = verificado, laranja = inferido, cinza = não encontrado
- Botão de exportar/compartilhar a ficha em CSV ou texto



Testing, Compliance and Quality Assurance

Responsáveis: Todos — vídeo: Isabelle

MAIOR LACUNA — prioridade máxima

O que a matéria pede

- Pitch de projeto: 10–15 slides + vídeo de até 3 minutos (todos participam)
- Business Model Canvas completo (9 blocos)
- Quadro de Valor: stakeholders, expectativas, métricas de negócio e de qualidade
- Arquitetura TOGAF no Archi — 4 visões obrigatórias + arquivo .archimate
- Entrega pelo Teams: apresentação + link do vídeo (no slide 1) + arquivo .archimate

O que fazer no SpecRadar

1. Pitch estruturado — estrutura obrigatória dos slides

- Slide 1: Nome SpecRadar + nomes e RMs de todos + link do vídeo
- Slide 2: Problema — 1h por versão, processo manual, imprecisão (dado documentado pela Ford)
- Slide 3: Escala do problema — não é exclusivo da Ford; citar empresas e mercados que enfrentam isso
- Slide 4: Solução — SpecRadar em 3 linhas: o que é, como funciona, para quem
- Slide 5: Benchmarking — iCarros, FIPE, Quatro Rodas, S&P Global não oferecem: extração por atributos + confidence score + repositório acumulativo
- Slide 6: Diferencial competitivo — confidence score por campo + sensor de demanda preditiva + repositório histórico
- Slide 7: Critérios de qualidade — precisão $\geq 95\%$ dos campos, tempo de resposta $< 10s$, disponibilidade $\geq 99\%$
- Slide 8: Riscos — hallucination de LLM, desatualização de dados, bloqueio de fontes web
- Slide 9: Métricas de sucesso — tempo de consulta, taxa de campos preenchidos vs. vazios, NPS do analista
- Slide 10: Roadmap — o que entrega na Sprint 1 vs. evoluções futuras (multimodal, PDF, imagem)

2. Business Model Canvas — 9 blocos para o SpecRadar

- Clientes: equipes de Product Marketing e Inteligência Competitiva da Ford
- Canais: app mobile (Expo) + interface web + API interna Ford
- Valor: eliminar 1h/versão de trabalho manual; dados padronizados, auditáveis e comparáveis
- Atividades-chave: manutenção do pipeline de extração; curadoria do banco histórico; sensor de demanda
- Recursos: infraestrutura cloud, modelo LLM, equipe de dados e desenvolvimento
- Parceiros: provedores de LLM (OpenAI/Anthropic), cloud (AWS/Azure/GCP)
- Custos: tokens de LLM por consulta, hospedagem cloud, manutenção do pipeline

- Receita/economia: redução de horas de analista × custo/hora × volume mensal
- Impactos de falha: decisões de posicionamento erradas, custo de retrabalho, risco reputacional

3. Quadro de Valor — stakeholders com métricas duplas (negócio + qualidade)

- PM Ford | espera: specs rápidas e corretas | entrega: ficha em <10s | métrica negócio: -80% tempo/versão | métrica qualidade: precisão ≥95%
- TI Ford | espera: integração segura e documentada | entrega: API REST + JWT + Swagger | métrica negócio: 0 incidentes de segurança | métrica qualidade: uptime ≥99%
- Liderança Ford | espera: redução de custo operacional | entrega: repositório histórico acumulativo | métrica negócio: horas economizadas/mês | métrica qualidade: disponibilidade da plataforma

4. Arquitetura TOGAF no Archi — 4 visões obrigatórias

- Visão de Arquitetura: objetivo estratégico (eliminar coleta manual) + requisitos funcionais e não-funcionais
- Arquitetura de Negócio: fluxo ATUAL (analista → busca manual → planilha) vs. fluxo COM SpecRadar (analista → app → ficha padronizada em <10s)
- Arquitetura de Sistema: componentes — app mobile, API gateway, serviço de extração, LLM, banco de specs, módulo de diff, sensor de demanda
- Arquitetura de Tecnologia: React Native + FastAPI + PostgreSQL + Redis cache + provider LLM + cloud hosting
- Entregar arquivo .archimate junto com a apresentação pelo Teams



O que a matéria pede (com pontuação)

1. Segurança de Entrada e Validação de Dados (20 pts)

- Validação de entradas: sanitização contra SQL Injection, XSS, command injection
- Normalização de parâmetros de API: marca, modelo, versão e atributos com padrões definidos
- Limitação de tamanho e formato: prevenir payload flooding e buffer overflow
- Tratamento seguro de erros: mensagens sem stack trace ou tecnologia interna

2. Autenticação e Autorização (20 pts)

- JWT com expiração, assinatura forte e renovação controlada
- RBAC: diferenciar analista (consulta/exporta) de admin (gerencia banco e configurações)

3. Proteção de APIs e Serviços (20 pts)

- HTTPS/TLS 1.2+ obrigatório em todos os endpoints
- Rate limiting e throttling: 60 req/min por usuário
- CORS configurado para aceitar apenas domínios autorizados
- Assinatura/verificação de integridade de payloads (5 pts extras)

4. Segurança de Dados e Privacidade (25 pts)

- Criptografia de dados sensíveis em repouso (specs no banco: AES-256)
- Política de retenção: dados com 12+ meses sem acesso são arquivados, não deletados
- Proteção contra exposição acidental: sem logs sensíveis, dumps ou endpoints não documentados

5. Monitoramento, Logs e Auditoria (15 pts)

- Logs estruturados com rastreabilidade: usuário + timestamp + ação + veículo consultado
- Monitoramento de eventos suspeitos: acessos indevidos, falhas repetidas, consultas massivas
- Trilha de auditoria para ações críticas: alterações de configuração, consultas em lote

O que implementar no SpecRadar

Validação de entrada com Pydantic (FastAPI)

- Classe SpecQuery: marca (str, 2–50 chars, só letras), modelo (str, 2–80), versao (str, 2–80), atributos (lista, 1–20 itens)
- Rejeitar qualquer input fora do padrão com 422 Unprocessable Entity
- Sanitizar inputs antes de passar ao LLM — prompt injection é o risco específico desta solução

JWT + RBAC

- Token de acesso: expiração 8h | Token de refresh: expiração 7d
- Role ANALYST: GET /specs, POST /specs/query, GET /specs/compare, GET /specs/history
- Role ADMIN: DELETE /specs/:id, PUT /specs/config, POST /specs/bulk-import
- Nenhum endpoint protegido aceita request sem Authorization: Bearer <token> válido

Rate limiting + HTTPS + CORS

- 60 requisições por minuto por usuário autenticado (evita scraping do banco)
- TLS 1.2+ obrigatório — certificate pinning no app mobile é diferencial
- CORS: aceitar apenas domínios da Ford (ford.com.br, ford.com) + localhost:3000 em dev

Logs de auditoria seguros

- Cada consulta registra: user_id (hash, não email), timestamp, marca+modelo consultados, campos solicitados
- Logs NUNCA contêm: email, nome, token JWT, stack trace, resposta completa da API
- Alertas automáticos: >100 consultas em 5 minutos por usuário → bloquear e notificar admin



Atenção — leia antes de qualquer código

A entrega de AI/ML foi desenhada especificamente para o Desafio 2 (VIN Share / retenção de clientes), com dados sintéticos em duas bases distintas. O SpecRadar resolve o Desafio 1. Antes de escrever qualquer código, Nicoli e Claudio devem perguntar ao professor da disciplina se o escopo pode ser adaptado para o Desafio 1 — e se sim, com qual formato de entregável.

O que a matéria pede (Desafio 2 — escopo original)

- Base 1 (histórico completo): segmentação comportamental com aprendizado não supervisionado (clustering)
- Base 2 (dados do momento da compra): classificação preditiva supervisionada — SEM variáveis futuras
- Quatro perfis hipotéticos: Fiel, Abandono, Esquecido, Econômico — verificar se dados sustentam
- Regra crítica: usar variáveis futuras na classificação = data leakage = trabalho invalidado
- Entregáveis: Jupyter Notebook (.ipynb) + relatório PDF com leitura executiva dos resultados

Opções se o professor autorizar escopo alternativo para Desafio 1

Opção A: Detecção de anomalia no banco de specs (Isolation Forest)

- Treinar modelo para detectar campos com valores improváveis (ex: torque de 900 Nm para carro popular)
- Resolve o problema de hallucination do LLM diretamente — aplicação ML real no Desafio 1
- Entregável: Notebook com análise + relatório mostrando anomalias detectadas no banco de validação

Opção B: Deduplicação semântica com sentence-transformers

- Usar embeddings para identificar que 'torque máximo 580 Nm' e 'torque de pico 580 newton-metro' são o mesmo dado
- Clustering semântico de atributos para normalizar variações de nomenclatura
- Demonstra embeddings, clustering e pipeline ML sem depender das bases do Desafio 2

Se o escopo for mantido no Desafio 2 (sem adaptação)

- O grupo produz a entrega de AI/ML completa para o Desafio 2 em paralelo com o SpecRadar
- Isso significa trabalhar os dois desafios simultaneamente — avaliar viabilidade com o grupo

- Nesse caso: pegar as bases com o professor, seguir o roteiro do enunciado e não usar variáveis futuras

Links

Sprint 1 | Testing, Compliance and Quality Assurance: [inserir link da entrega no Teams]

Repositório do projeto: [inserir link do GitHub]

Swagger / documentação da API: [inserir link após deploy]

Vídeo pitch: [inserir link após gravação — colocar também no Slide 1]

Arquivo Archi (.archimate): [anexar junto com a apresentação pelo Teams]

Checklist final — antes de enviar

- Apresentação com 10–15 slides, nome e RM de todos no slide 1, link do vídeo no slide 1
- Vídeo pitch de até 3 minutos com todos os integrantes participando
- Arquivo .archimate gerado no Archi com as 4 visões TOGAF
- Tudo enviado junto pelo Teams até 24/05/2026